

Doubly Linked Lists

A Second Pointer, and the Two-Way Header List

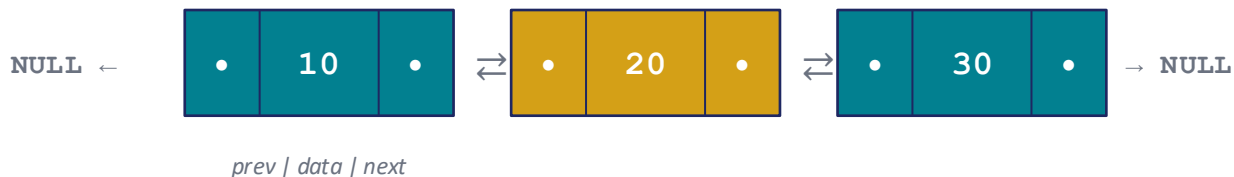
Unit II • Lecture 19



The node: one more pointer, twice the links to manage

```
struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};
```

- prev: address of the node BEFORE this one, or NULL if this is the first.
- next: address of the node AFTER this one, or NULL if this is the last — unchanged from before.
- Every node except the two ends has BOTH neighbours addressable directly.



insertFront(): three links, not one

```
struct Node *insertFront(struct Node *head,
                        int value)
{
    struct Node *node = malloc(sizeof(*node));
    if (node == NULL) exit(1);
    node->data = value;
    node->prev = NULL;
    node->next = head;

    if (head != NULL)
        head->prev = node;

    return node;
}
```

- node->next = head: same as the singly linked version.
- node->prev = NULL: the new node is now the first, so nothing comes before it.
- head->prev = node: the NEW link — the OLD head must now look back at the new node. Easy to forget.

deleteNode(): the genuine advantage of prev

Lecture 18's deleteAtPosition had to WALK from head to find the node before the target — $O(n)$. Today: if you already hold a pointer to the target, prev gets you there directly — $O(1)$.

```
struct Node *deleteNode(struct Node *head,
                        struct Node *target)
{
    if (target->prev != NULL)
        target->prev->next = target->next;
    else
        head = target->next; /* target was head */

    if (target->next != NULL)
        target->next->prev = target->prev;

    free(target);
    return head;
}
```

- Two "stitch around" steps, one for each side — no search loop anywhere.
- Each side has its own guard: target might be the first node (no prev) or the last (no next).

Traversal now works in both directions

```
/* forward: exactly as before */
for (p = head; p != NULL; p = p->next)
    printf("%d ", p->data);

/* backward: walk to the end, then
   follow prev back to the start */
while (p->next != NULL) p = p->next;
while (p != NULL) {
    printf("%d ", p->data);
    p = p->prev;
}
```

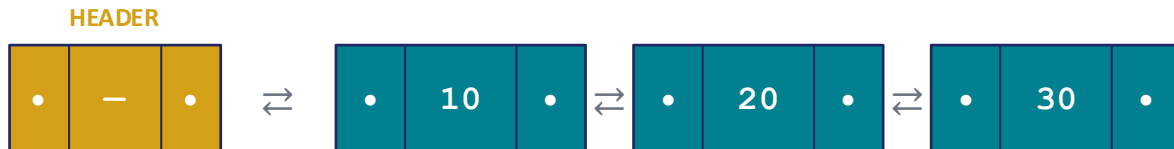
- Forward traversal is UNCHANGED from Lecture 16.
- Backward traversal needs a starting point at the LAST node first — without a tail pointer, that itself costs $O(n)$.
- Once at the last node, walking back is free: each prev is a direct hop.

Verified output for 10,20,30: Forward: 10->20->30->NULL Backward: 30->20->10->NULL

The Two-Way Header List: one extra node, fewer special cases

The idea

Add ONE permanent HEADER node that is never deleted and never holds real data. It always exists — so "is the list empty?" and "is this the first node?" stop being special cases.



header->next wraps to 10;

last node's next wraps to header (circular), or NULL (non-circular)

- "List empty?" becomes `header->next == header` (or `NULL`) — no head == `NULL` check anywhere else.
- Insert/delete at the front is now IDENTICAL in shape to insert/delete anywhere else — the header always plays the role of "the node before."

Full trace: build, traverse, delete — verified

Call	Result
<code>insertEnd x3: 10, 20, 30</code>	Forward: 10 -> 20 -> 30 -> NULL
<code>traverseBackward</code>	Backward: 30 -> 20 -> 10 -> NULL
<code>insertFront(5)</code>	Forward: 5 -> 10 -> 20 -> 30 -> NULL
<code>deleteNode(target = node holding 20)</code>	Forward: 5 -> 10 -> 30 -> NULL
<code>deleteNode(target = head, i.e. 5)</code>	Forward: 10 -> 30 -> NULL

Compiled and run with AddressSanitizer — zero leaks. Full source in the .txt file.

Singly vs Doubly Linked — what you paid for what you got

Aspect	Singly Linked	Doubly Linked
Pointers per node	1 (next)	2 (prev, next)
Traversal	Forward only	Forward AND backward
Delete a KNOWN node	$O(n)$ — must find predecessor	$O(1)$ — prev gives it directly
Memory per node	Lower	Higher (extra pointer)
Insert/delete bookkeeping	Up to 2 links to fix	Up to 4 links to fix

A second pointer changes everything

- Every node gets a prev pointer alongside next — enabling backward traversal, which a singly linked list cannot do at all.
- Insertion needs up to 4 links fixed instead of 2; every new link needs its own NULL guard for the list's two ends.
- The real win: deleting a node you already hold a pointer to is $O(1)$ — no predecessor search, unlike Lecture 18.
- A two-way HEADER node removes most front/empty special cases — there is always a "node before" to point at.

EXIT TICKET Draw a 4-node doubly linked list (1,2,3,4). Trace deleteNode on the node holding 3 by hand — exactly which TWO pointer fields change value, and in which two nodes?